XG Boost : Extreme Gradient Boosting.

ABDM healthcare Data of Indian

GBM ⇒ Gradient Boosting Machine

↳ Large dataset

↳ Tabular data

1) Algorithmic optimization (Gradient Boosting, Sparsity aware split finding (missing value))

2) System optimization

Dosage	Drug effectiveness	Residuals
10	-7	-7.5
???	-3	-3.5
21	7	6.5
25	8	7.5
5	-5	-5.5
???	-2	-2.5

5	-5	-5.5
10	-7	-7.5
21	7	6.5
25	8	7.5

all value table

??	-3	-3.5
??	-2	-2.5

all missed value table

default Path.

Dosage < 7.5

N Y

-7.5, 6.5, 7.5 | -5.5

$IG_{right} > IG_{left}$

default Path.

$IG_{right} : \{-5.5, -3.5, -2.5\}$

$IG_{left} = \{-7.5, 6.5, 7.5, -3.5, -2.5\}$

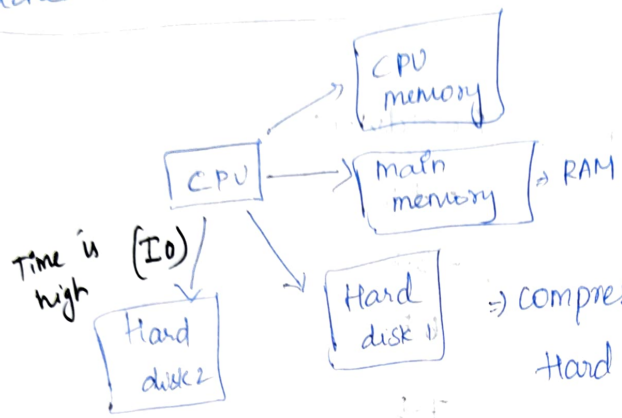
Parallel learning

x_1
5
3
8
24
2

million of Samples distributed among the (m) computers to sort it out

Cache Aware Access

It cleverly



⇒ compress and store data in Hard disk

Blocks for out of core computation

Sharding ⇒ Sharing/ Data in different DB
distributing

chest pain	Good blood circulation	Blocked arteries	Heart disease	residual
N 0	N 0	N 0	N 0	0 - 0.66 -0.66
Y 1	Y 1	Y 1	Y 1	1 - 0.66 +0.34
Y 1	Y 1	N 0	Y 1	1 - 0.66 0.34
Y 1	N 0	N 0	Y 1	1 - 0.66 0.34
Y 1	N 0	Y 1	Y 1	1 - 0.66 0.34
N 0	Y 1	N 0	N 0	0 - 0.66 -0.66

Gradient Boosting for Classification

* log odds
* convert to probability

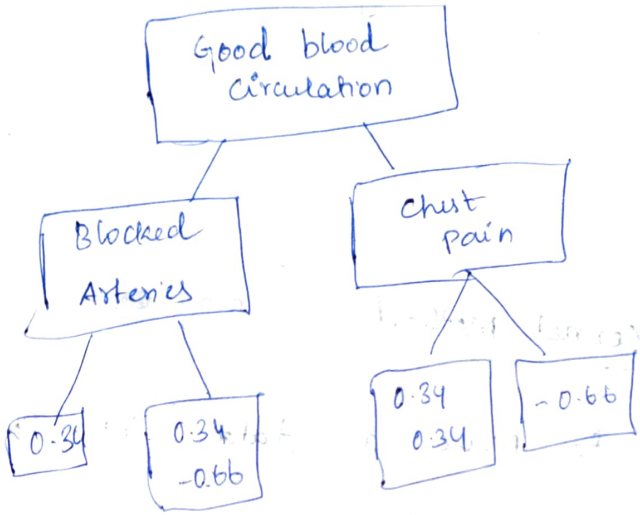
Train a gradient boosting tree for classification

Step 1: Initial prediction

1) $\log(\text{odds}) = \log(4/2) \approx 0.7$

2) Convert into probability

$$P = \frac{e^{\log(\text{odds})}}{1+e^{\log(\text{odds})}} = \frac{e^{\log(4/2)}}{1+e^{\log(4/2)}} = 0.66$$



3) predict residuals convert into log odds

$$\frac{\sum_i \text{Residual}}{\sum\left[\text{Prev } P_i \times (1-\text{Prev } P_i)\right]}$$

for 1st leaf $= \dfrac{0.3}{0.66 \times (1-0.66)} \approx 1.43$

for 2nd leaf $= \dfrac{0.34 + -0.66}{(0.66 \times (1-0.66)) + (0.66 \times (1-0.66))} \approx -0.95$

$= \dfrac{-0.32 - 0.4}{0.21 + 0.21} = \dfrac{-0.4}{0.42}$

3rd leaf = $\frac{0.3 + 0.3}{(0 + (1-0.7)) + (0.7 + (1-0.7))}$

$= \frac{0.6}{0.42}$

Final log(odds) = log(odds) + $\sum_{b=1}^{B} \lambda \cdot \log(\text{odds})_b$

Kernel method ⇒ fit non linear model

It is possible to move a data into higher dimension to fit a data

$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots$

$x \in \mathbb{R}^2 \quad \theta \Rightarrow \mathbb{R}^2 \quad \theta \Rightarrow \mathbb{R}^d$

$p >> d$

$x \in \mathbb{R} \quad \underbrace{\phi(x)}_{\text{function move data into 2D into 4D}} = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \Rightarrow \theta \Rightarrow \mathbb{R}^p$ features

$\theta_j := \theta_j - \alpha \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$

$= \theta_j + \alpha \sum_{i=1}^{n} (y^{(i)} - \underbrace{h_\theta x^{(i)}}_{\theta^T(\phi x^{(i)})}) \cdot x_j^{(i)}$

$= \theta_j + \alpha \sum_{i=1}^{n} (y^{(i)} - \underbrace{\theta^T}_{\theta \in \mathbb{R}^p}(\phi x^{(i)})) \phi(x^{(i)})$

before transformation attribute

After transformation feature

$f(x) \}$ feature map

↓ next